

Final Project Report Checklist & Grade Breakdown

1. Problem Definition (2 Points)

- **What to include:** A 1-page outline covering the core problem requirements, application domain, and project motivations.

2. Application Benefits (4 Points)

- **What to include (2 pages):** Highlight your app's advantages by comparing it directly to two existing real-world systems. Include proper references/URLs for both.

3. Requirements Elicitation (10 Points)

- **Functional Requirements (4 pts):** List the implemented functional requirements for your two exact user roles (Hospital Staff and Donors), naming and explaining each.
- **Software Qualities (6 pts):** Provide two concrete examples for *Correctness*, *Time-efficiency*, and *Robustness* across your user roles. *Note: Each use case can only be used once.*

4. Functional Requirements Specification (12 Points)

- **Use Case Diagrams (6 pts):** Provide complete use case diagrams with actors and all use cases for both user roles.
- **Activity Diagrams (6 pts):** Write detailed activity diagrams for your two most complex use cases (those involving secondary flows).

5. Software Design (29 Points)

- **MVC Architecture (8 pts):** Map out the MVC architecture according to your chosen web framework and list 3 specific benefits (qualities) it brings to your app.
- **Design Patterns (16 pts):** Detail the *Observer* and *Simple Factory* patterns. Include complete class diagrams (attributes with data types, method prototypes, and method algorithms) following GoF principles.
- **Whole Class Diagram (5 pts):** Provide the full class diagram integrating both design patterns into the overall software.

6. Software Implementation (28 Points)

- **Code Structure Screenshot (2 pts):** A clear screenshot showing the top-level file structure within your web framework.
- **Deployment Diagram (3 pts):** A hardware configuration diagram indicating supported web browsers, servers, and database solutions.
- **GitHub Repository (17 pts):** A link to your GitHub repo showing equal contribution, proper commit logs, and consistent commit messages (TaskID + description).
- **Live Deployment URL (6 pts):** The public, runnable URL where your web app is hosted online.

7. Technical Documentation (5 Points)

- **Languages (1 pt):** A list of all programming languages used.
- **Reused Code (1 pt):** A list of reused snippets/algorithms along with their source URLs.
- **Tools & Environments (3 pts):** A list of software tools/environments used, noting their specific benefits for your application.

8. Acceptance Testing (10 Points)

- **Correctness Testing (4 pts):** Four test cases with screenshots showing both inputs and outputs. *(Each use case tested only once).*
- **Robustness Testing (4 pts):** Four test cases with screenshots of inputs and outputs handling edge cases/errors.
- **Time-Efficiency Testing (2 pts):** Testing two specific functions with screenshots and a clear explanation of how the time was measured.

*Total Project Score: **100 Points***

Summary of Diagrams to Create

1. **Use Case Diagram - Donor Role (Section 4a)**
 - **What it looks like:** A system boundary box representing your web app, with the "Donor" stick figure actor on the left. Connected via lines/arrows are ovals representing use cases like *Register/Login*, *View Active Alerts*, *Complete Pre-Screening Form*, and *Update Availability*.
2. **Use Case Diagram - Hospital / Blood Bank Role (Section 4a)**
 - **What it looks like:** A separate system boundary box or shared system context with the "Hospital Staff" actor on the left, connected to ovals such as *Staff Login*, *Trigger Emergency Broadcast*, and *View Live Response Tracker*.
3. **Activity Diagram - Complex Use Case 1 (Emergency Dispatch & Notification Flow) (Section 4b)**
 - **What it looks like:** A flowchart tracking the step-by-step sequential workflow: Hospital submits request → Backend triggers spatial PostGIS query → Filtering matching donors → Conditional check (Are donors available?) → Multi-threaded dispatch via FCM/Twilio → End.
4. **Activity Diagram - Complex Use Case 2 (Pre-Screening & Eligibility Triage Flow) (Section 4b)**
 - **What it looks like:** A flowchart showing: Donor clicks push alert link → Opens digital questionnaire → User inputs answers (alcohol, travel, medications) →

Conditional gateway (Did they pass checks?) → Branch A (Approved: Generate routing/QR) vs. Branch B (Deferred: Show delay message).

5. **Pattern-Specific Class Diagrams** (*Section 5b*)

- **Observer Pattern Class Diagram:** Shows the Subject interface/class (e.g., EmergencyBroadcastManager) and Observer interface/class (e.g., DonorNotificationListener) demonstrating how donors are notified dynamically when an emergency request state changes.
- **Simple Factory Pattern Class Diagram:** Shows a creator class (e.g., NotificationFactory) that instantiates different concrete notification objects (e.g., PushNotification vs. SMSNotification) based on input parameters.

6. **Whole Software Class Diagram** (*Section 5c*)

- **What it looks like:** A comprehensive architectural class map combining all database models (User, Institution, EmergencyRequest), controllers, services, and integrating the *Observer* and *Simple Factory* classes together.

To master how to design these exact blueprints, you can watch this comprehensive [UML class diagrams tutorial](#) to understand how to properly format attributes, methods, and visibility markers.

<https://www.youtube.com/watch?v=6XrL5jXmTwM&start=1>